Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

07

MODULE 07

Git Branches and Merging

Work on ideas in parallel, combine them safely, and resolve conflicts calmly.

MODULE 07 · LESSON 7.1

Understanding branches

فهم الفروع

Learning objectives

- Define a branch.
- Explain why branches exist.
- Name feature and bug-fix branches.
- Explain how commits relate to branches.

Prerequisites

Module 6.

Introduction

Imagine trying a risky idea without touching your working project. Branches make that possible.

Explanation

A **branch** is a movable label that points to a commit. Making one costs almost nothing, because Git does not copy files.

main is the default branch. Keep it stable and working.

A **feature branch** holds new work, for example `feature/homepage`. A **bug-fix branch** holds a repair, for example `fix/nav-spacing`.

Parallel development. You can work on a feature while a teammate fixes a bug, each on a separate branch, without disturbing each other.

Commits and branches. When you commit, the current branch label moves forward to the new commit. **HEAD** tells Git which branch you are on. When ready, you **merge** the branch back into main (Lesson 7.2).

Real-world analogy

> A branch is a photocopy of a draft you scribble on. If the idea is good, you copy the changes into the original. If not, you throw the copy away.

Diagram

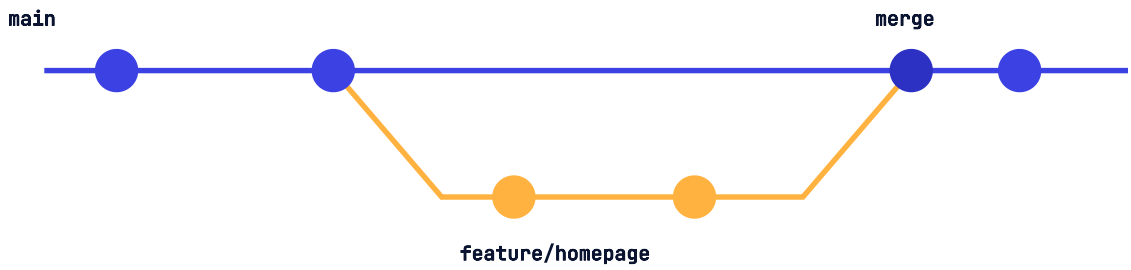


Figure 1 · A feature branch and a merge

Commands

No commands yet. Figure 1 shows a feature branch leaving main and returning through a merge.

Common mistakes and solutions

Thinking a branch is a full copy.	It is only a pointer to a commit, so it is fast and small.
Working on main directly.	Create a branch for each task.
Long-lived branches.	Merge often to avoid painful conflicts.

Practical exercise

> Draw on paper: main with three commits, a branch leaving after the second, two commits on it, then a merge.

Expected result

Your drawing matches Figure 1.

Knowledge check

1. What is a branch?

- ○ A GitHub account
- ○ A pointer to a commit
- ○ A copy of all files

2. What is a feature branch for?

- ○ New work
- ○ Deleting main
- ○ Installing Git

3. What does HEAD show?

- ○ Your email
- ○ The branch/commit you are on
- ○ The remote URL

Answers

- A pointer to a commit — Branches are lightweight pointers.
- New work — It isolates new work.
- The branch/commit you are on — HEAD is your current position.

Summary

Branches let you work in parallel safely. Commits move the current branch forward, and merges combine branches.

Key vocabulary

Branch	Movable pointer to a commit
Feature branch	Branch for new work
Merge	Combine branches

Next lesson

Lesson 7.2 — Creating and merging branches.

MODULE 07 · LESSON 7.2

Creating and merging branches

إنشاء الفروع ودمجها

Learning objectives

- List, create and switch branches.
- Merge a branch into main.
- Delete a merged branch.

Prerequisites

Lesson 7.1 and a clean working tree.

Introduction

Now you create a branch, do some work, and bring it back.

Explanation

`git branch` lists local branches. The star marks the current one.

`git switch -c feature/homepage` creates the branch and switches to it. `-c` means “create”.

Edit `index.html`, then `git add index.html` and `git commit -m "Add welcome section"`. The commit lands on the feature branch only.

`git switch main` returns to main. Your edit disappears from the folder because main does not have it yet.

`git merge feature/homepage` brings the commits into main. If main has not changed since you branched, Git just moves the main label forward: **Fast-forward**. If both changed, Git creates a merge commit.

`git branch -d feature/homepage` deletes the branch label after merging. `-d` refuses if it is not merged, which protects you.

To share the branch: `git push -u origin feature/homepage`.

Real-world analogy

> Switching branches is changing the film in a camera: the camera (your folder) stays, the picture it shows changes.

Diagram



Figure 1 · Illustration of the expected output

Commands

`git branch`
`git switch -c feature/homepage`
`git switch main`
`git merge feature/homepage`
`git branch -d feature/homepage`

Options: `-c` create; `-d` delete if merged. Figure 1 shows the merge and clean-up.

Common mistakes and solutions

"Your local changes would be overwritten"	Commit your changes before switching.
Merging into the wrong branch.	Switch to the branch that should receive the work first.
"not fully merged" on delete	Merge first, or confirm you really want `-D` (force).

Practical exercise

> Create `feature/homepage`, add a welcome paragraph, commit, switch to main, merge, and delete the branch.

Expected result

main contains the paragraph and `git branch` lists only main.

Knowledge check

1. Which command creates and switches?

- ○ git push name
- ○ git switch -c name
- ○ git merge name

2. What is Fast-forward?

- ○ Git moves the label ahead without a merge commit
- ○ A deleted file
- ○ A GitHub setting

3. Which branch receives the work in a merge?

- ○ Always the remote
- ○ The one you are on
- ○ The other one

Answers

- git switch -c name — -c creates the branch.
- Git moves the label ahead without a merge commit — Happens when main has not diverged.
- The one you are on — Merge into the current branch.

Summary

Create a branch with switch -c, commit on it, switch to main, merge, then delete the branch.

Key vocabulary

git switch	Change branches
Fast-forward	Merge by moving the label ahead
git branch -d	Delete a merged branch

Next lesson

> **Lesson 7.3 — Merge conflicts.**

MODULE 07 · LESSON 7.3

Merge conflicts

تعارضات الدمج

Learning objectives

- Explain why conflicts happen.
- Read conflict markers.
- Resolve with the VS Code Merge Editor.
- Finish the merge.

Prerequisites

Lesson 7.2.

Introduction

A conflict is not an error. It is Git asking you to decide.

Explanation

Why. Two branches changed the same lines differently. Git cannot guess which to keep.

Identify. The merge stops with `CONFLICT (content): Merge conflict in index.html`. `git status` lists the file under “Unmerged paths” as “both modified”.

Markers. Git writes both versions into the file. Between `<<<<<<< HEAD` and `=======` is your current branch. Between `=======` and `>>>>>>> feature/homepage` is the incoming branch (Figure 1).

Resolve. In VS Code the file shows buttons above the conflict: **Accept Current**, **Accept Incoming**, **Accept Both**, or open **Resolve in Merge Editor** for a three-pane view. Choose or combine, and make sure no markers remain.

Test. Open the page and check it still works.

Finish. Run `git add index.html` then `git commit` (or `git merge --continue`). Git proposes a merge message.

Escape hatch. `git merge --abort` cancels the merge and returns to the state before it.

Real-world analogy

> Two editors rewrote the same sentence. The author must read both and write the final sentence.

Diagram

Illustration · Git Bash (not a real screenshot)

```
$ git merge feature/homepage
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

<<<<<<< HEAD
<h1>Hello, World!</h1>
=======
<h1>Welcome to my site</h1>
>>>>>>> feature/homepage
```

Figure 1 · Illustration: a file with conflict markers

Commands

`git merge feature/homepage`
`git status`
`git add index.html`
`git commit`
`git merge --abort`

Figure 1 shows the conflict markers in the file.

Common mistakes and solutions

Leaving the markers in the file.	Search for `<<<<<<<` before committing.
Panicking.	Use `git merge --abort` and try again calmly.
Not testing after resolving.	Always open the result.

Practical exercise

> Read the lab in this module and resolve the conflict it creates.

Expected result

The file has one heading, no markers, and `git status` is clean after the merge commit.

Knowledge check

1. Why do conflicts happen?

- ○ GitHub is offline
- ○ Two branches changed the same lines
- ○ Git is broken

2. What does HEAD mean between the markers?

- ○ Your current branch version
- ○ The remote
- ○ The deleted version

3. How do you cancel a conflicted merge?

- ○ git push
- ○ git merge –abort
- ○ git delete

Answers

- Two branches changed the same lines — Git cannot choose automatically.
- Your current branch version — The first block is your side.
- git merge –abort — It restores the pre-merge state.

Summary

Conflicts mean Git needs your decision. Edit the file, remove markers, test, add, and commit, or abort.

Key vocabulary

Conflict	Same lines changed differently
Conflict markers	Lines with `<<<<<<<`, `=======`, `>>>>>>>`
Merge Editor	VS Code three-pane resolver

Next lesson

Module 8 — Clone, fetch and pull.

MODULE 07

Practical lab 7 · Create, merge and resolve a conflict

1. Make sure `git status` is clean on main.
2. `git switch -c feature/homepage`. Change the heading line to `<h1>Welcome to my site</h1>`. Commit: `Update heading on feature branch`.
3. `git switch main`. Change the same line to `<h1>Hello, World!</h1>`. Commit: `Update heading on main`.
4. Run `git merge feature/homepage`. A conflict appears.
5. Open `index.html`, keep `<h1>Welcome to my site</h1>` and remove all markers.
6. `git add index.html`, then `git commit`. Delete the branch with `git branch -d feature/homepage`.

Check	Pass if…
Conflict	You saw CONFLICT in index.html
Resolution	No markers remain
History	`git log --oneline` shows the merge commit

Evaluation: 3 of 3 = pass.

Module 7 quiz (pass mark: 70%)

1. Which command lists branches?

- ○ git log
- ○ git diff
- ○ git branch

2. How do you create and switch?

- ○ git add name
- ○ git switch -c name
- ○ git merge name

3. What does the merge do?

- ○ Combines another branch into the current one
- ○ Deletes a branch
- ○ Uploads to GitHub

4. What are `<<<<<<<` markers?

- ○ Passwords
- ○ Conflict markers
- ○ Comments

5. Which command cancels a merge?

- ○ git reset
- ○ git push
- ○ git merge –abort

Answers

- git branch — The star marks the current one.
- git switch -c name — -c means create.
- Combines another branch into the current one — You merge into the current branch.
- Conflict markers — They mark the two versions.
- git merge –abort — It restores the state before the merge.